



Sistema de Gestión Club Shaolin

Trabajo de Fin de Grado (TFG)
Grado Superior en Desarrollo de Aplicaciones Web (DAW)
IES Hermenegildo Lanz
Francisco Javier Fernández Hernández



ÍNDICE

GESTIÓN CLUB SHAOLIN.....	3
1. Introducción.....	3
2. Objetivos del Proyecto.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos	3
3. Características del Sistema	4
4. Tecnologías Utilizadas	4
4.1 Frontend	4
4.2 Backend.....	4
4.3 Infraestructura	4
5. Arquitectura del Sistema	5
5.1 Flujo de funcionamiento	5
6. Estructura del Proyecto.....	6
7. Modelo Entidad-Relación (E-R).....	7
8. Diseño de las Tablas (Listado de Atributos)	8
9. Roles del Sistema.....	8
10. Casos de Uso	9
11. Despliegue del Sistema.....	9
12. Capturas de Pantalla	10
13. Conclusiones.....	13
14. Referencias	13

GESTIÓN CLUB SHAOLIN

1. Introducción

El proyecto “Sistema de Gestión del Club Shaolin” ha sido desarrollado como Trabajo de Fin de Grado (TFG) del ciclo formativo de Desarrollo de Aplicaciones Web (DAW).

El objetivo de este documento es detallar el funcionamiento del sistema, las tecnologías empleadas, la arquitectura utilizada, el modelo de datos, así como los principales casos de uso y el proceso de despliegue.

El sistema desarrollado consiste en una aplicación web orientada a la digitalización y gestión administrativa del gimnasio Club Shaolin Las Gabias. Este proyecto surge con la finalidad de modernizar la gestión tradicional del centro, basada en papel y hojas de cálculo, centralizando el control de alumnos, asistencias, eventos, pagos y licencias en una plataforma web intuitiva, eficiente y accesible.

2. Objetivos del Proyecto

2.1 Objetivo general

Desarrollar una aplicación web que permita gestionar de forma eficiente todas las operaciones administrativas de un gimnasio de artes marciales.

2.2 Objetivos específicos

- Digitalizar la gestión de alumnos.
- Automatizar el control de pagos y cuotas.
- Facilitar la organización de eventos.
- Implementar un sistema de roles con control de acceso.
- Garantizar la seguridad mediante autenticación.
- Diseñar una interfaz responsive accesible desde cualquier dispositivo.

3. Características del Sistema

El sistema desarrollado incluye las siguientes funcionalidades principales:

- Gestión de Alumnos: Registro completo de expedientes, datos personales e historial.
- Control de Pagos y Cuotas: Automatización y seguimiento de mensualidades y licencias.
- Gestión de Eventos: Organización e inscripción de alumnos en exámenes, concentraciones y competiciones.
- Licencias: Gestión de estados de pago de licencias.
- Seguridad: Autenticación basada en JWT y control de acceso por roles.
- Diseño Responsivo: Adaptación a dispositivos móviles, tablets y ordenadores.

4. Tecnologías Utilizadas

4.1 Frontend

- React 19
- TypeScript
- Vite
- React Router 7
- Vanilla CSS

4.2 Backend

- Python / Flask
- Flask-SQLAlchemy
- Flask-Migrate
- Flask-JWT-Extended

4.3 Infraestructura

- MySQL
- Nginx
- Docker & Docker Compose

5. Arquitectura del Sistema

El sistema desarrollado sigue una arquitectura basada en servicios desacoplados mediante contenedores, donde cada componente funciona de manera independiente y se ejecuta dentro de contenedores de Docker. Esta arquitectura permite una mayor escalabilidad, mantenimiento sencillo y facilidad de despliegue en distintos entornos.

El sistema está compuesto por los siguientes elementos:

- Frontend: Aplicación web desarrollada con React y TypeScript. Es la interfaz con la que interactúan los usuarios (administradores, ayudantes y alumnos).
- Backend: API REST desarrollada con Flask (Python), encargada de gestionar la lógica de negocio, autenticación, control de acceso y procesamiento de datos.
- Base de Datos: Sistema MySQL que almacena toda la información del sistema (usuarios, asistencias, eventos, pagos, licencias, etc.).
- Servidor Web: Nginx actúa como proxy inverso, gestionando las peticiones entrantes y redirigiéndolas al frontend o al backend, según corresponda.
- Contenedores: Todos los servicios están dockerizados y orquestados mediante Docker Compose, lo que permite levantar todo el sistema de forma automática y consistente.

5.1 Flujo de funcionamiento

El flujo de interacción del sistema es el siguiente:

Usuario → Frontend → API (Backend) → Base de Datos

1. El usuario accede a la aplicación desde su navegador.
2. El frontend (React) muestra la interfaz y envía peticiones HTTP al backend.
3. El backend (Flask) procesa la solicitud, aplica la lógica de negocio y valida permisos mediante JWT.
4. Si es necesario, el backend interactúa con la base de datos MySQL.
5. La respuesta se devuelve al frontend, que actualiza la interfaz de usuario dinámicamente.

Este modelo permite separar claramente responsabilidades entre cliente y servidor, mejorando la organización del código y la seguridad del sistema.

6. Estructura del Proyecto

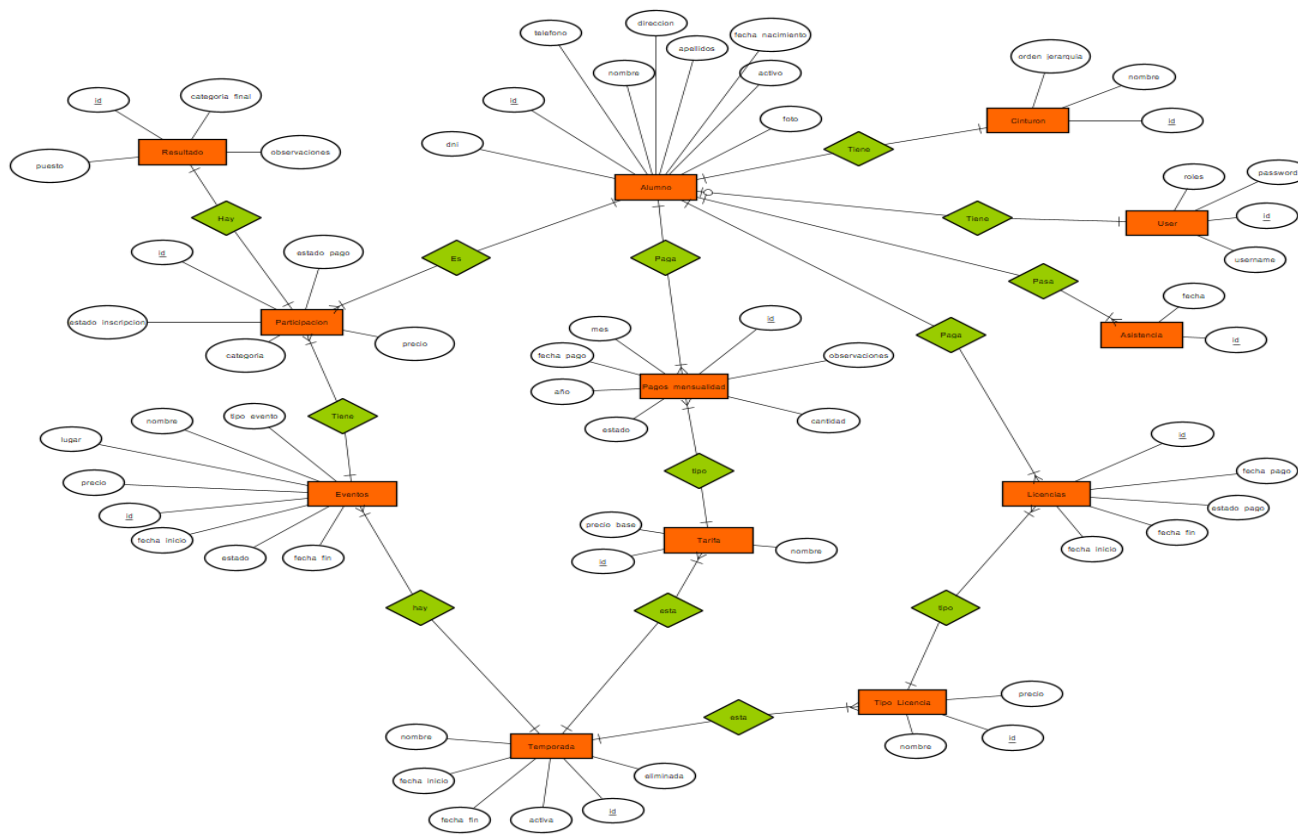
El proyecto se organiza en diferentes carpetas que separan claramente cada parte del sistema, facilitando su mantenimiento y escalabilidad:

```
.
├── backend/      # API REST desarrollada con Flask
│   ├── app.py    # Punto de entrada de la aplicación
│   ├── models/   # Modelos de la base de datos (SQLAlchemy)
│   ├── controllers/ # Lógica de negocio y rutas
│   └── migrations/ # Historial de migraciones de la base de datos
├── frontend/     # Aplicación web SPA (React)
│   └── app/
│       ├── src/   # Componentes, páginas, hooks y servicios
│       └── public/ # Archivos estáticos
├── nginx/        # Configuración del servidor web y proxy inverso
└── docker-compose.yml # Definición y orquestación de los contenedores
```

Descripción de las carpetas

- backend/
Contiene toda la lógica del servidor. Incluye la API REST, modelos de base de datos, controladores y migraciones. Es el núcleo funcional del sistema.
- frontend/
Incluye la aplicación cliente desarrollada en React. Aquí se gestiona la interfaz de usuario, navegación, consumo de API y experiencia del usuario.
- nginx/
Contiene la configuración del servidor Nginx, que actúa como intermediario entre el cliente y los servicios internos (frontend y backend).
- docker-compose.yml
Archivo encargado de definir y coordinar todos los servicios del sistema (frontend, backend, base de datos y nginx), permitiendo su ejecución conjunta mediante Docker.

7. Modelo Entidad-Relación (E-R)



El modelo de datos del sistema ha sido diseñado con el objetivo de representar de forma estructurada toda la información necesaria para la gestión del gimnasio, garantizando la integridad, coherencia y facilidad de acceso a los datos.

Para ello, se ha utilizado un modelo entidad-relación (ER) que define las principales entidades del sistema y las relaciones existentes entre ellas. Este modelo permite organizar la información de manera lógica y eficiente, facilitando su posterior implementación en una base de datos relacional.

Relaciones

- **Alumno** se relaciona con Participación, Pago del Mes, Pago de Licencia, Asistencias, Usuarios y Cinturón.
- **Temporada** se relaciona con Tipo de Licencia, Tarifa y Eventos.
- **Participación** se relaciona con Resultado y con Eventos.
- **Licencia** se relaciona con Tipo de Licencia.
- **Pago del Mes** se relaciona con Tarifa.

8. Diseño de las Tablas (Listado de Atributos)

User => id, username, password, roles

Alumno => id, user_id, cinturon_id, nombre, apellidos, dni, fecha_nacimiento, teléfono, dirección, foto, activo

Cinturón => id, nombre, orden

Asistencia => id, fecha, alumno_id

Temporada => id, nombre, fecha_inicio, fecha_fin, activa, eliminada (para eliminado lógico)

Tarifa_Mensual => id, nombre, precio_base, temporada_id

Tipo_Licencia => id, nombre, precio, temporada_id

Licencia => id, alumno_id, tipoLicencia_id, fecha_pago, fecha_inicio, fecha_fin, estado_pago

Pago_Mensualidad => id, alumno_id, mes, año, tarifa_id, cantidad, estado, fecha_pago, observaciones

Evento => id, nombre, tipo, fecha_inicio, fecha_fin, lugar, precio_inscripción, estado, temporada_id

Participación => id, evento_id, alumno_id, categoría, estado_inscripción, estado_pago, precio

Resultado => id, participación_id, puesto, categoria_final, observaciones

9. Roles del Sistema

El sistema tiene un control de acceso basado en roles:

- **Administrador:**
Gestión completa del sistema
- **Ayudante (Cinturones Negros profesores):**
Gestión parcial
- **Alumno**
Acceso a su información personal

10. Casos de Uso

Casos por rol:

Administrador:

- Gestionar Alumnos
- Gestionar Eventos
- Gestionar Asistencias
- Gestionar Mensualidades
- Gestionar Licencias
- Gestionar Temporadas, Usuarios, Cinturones

Ayudante:

- Gestionar Alumnos
- Gestionar Eventos
- Gestionar Asistencias

Alumnos:

- Información de Pagos de Mensualidades y Licencia
- Información de su Licencia

11. Despliegue del Sistema

Para levantar el entorno completo (Backend, Frontend, Base de Datos y Nginx) de manera automática:

1. Clonar el repositorio:

- HTTPS

```
git clone https://github.com/fjfh06/TFG-DAW.git
cd TFG-DAW
```

- Recomendado (SSH)

```
git clone git@github.com:fjfh06/TFG-DAW.git
cd TFG-DAW
```

2. Configurar variables de entorno: Crea un archivo **.env** en la raíz del proyecto (puedes basarte en **.env.ejemplo**) con las credenciales necesarias.

3. Levantar servicios

```
docker compose up --build -d
```

4. Acceso

- Aplicación Web: `http://localhost`
- API (Backend): `http://localhost/api` (vía Nginx)
- Base de Datos (Administración opcional): PHPMyAdmin en el puerto configurado.

12. Capturas de Pantalla

Dashboard administrador

Club Shaolin Las Gabias

Panel de Control

Hola, **Francisco Javier** 🙌

Bienvenido al panel de control del Club Shaolin Las Gabias. Estás gestionando la temporada: **Temporada 25/26**

Alertas y Seguimiento

- Sin Licencia** (3)
 - Noelia Nogueras
 - Patricia Ocón
 - Ángela Gongost
- Sin Pagar Mes Actual** (1)
 - Patricia Ocón
- Deudas de Meses Anteriores** (7)
 - Gemma Serrano Fernández
 - Francisco Javier Fernández Hernández
 - Noelia Nogueras

TEMPORADA: **Temporada 25/26 (Actual)**

Francisco Javier
ADMINISTRADOR

Cerrar Sesión

Dashboard del administrador donde tiene información relevante sobre los alumnos

Dashboard ayudante

Club Shaolin Las Gabias

Panel de Control

Bienvenido al panel, estás en la temporada: **Temporada 25/26**

Noelia Nogueras

Estado: Vigente Último acceso: Hoy

Monitor Fiscal

ESTADO DE CUOTAS ANUALES

MENSUALIDADES PENDIENTES

HISTORIAL RECIENTE

4 abril RECIBO OK 30.00€

Ficha Federativa
Licencia no Emite

Consulte al shifu para activar su cuota definitiva

Panel de Control

Pasar Asistencia

Gestión Alumnos

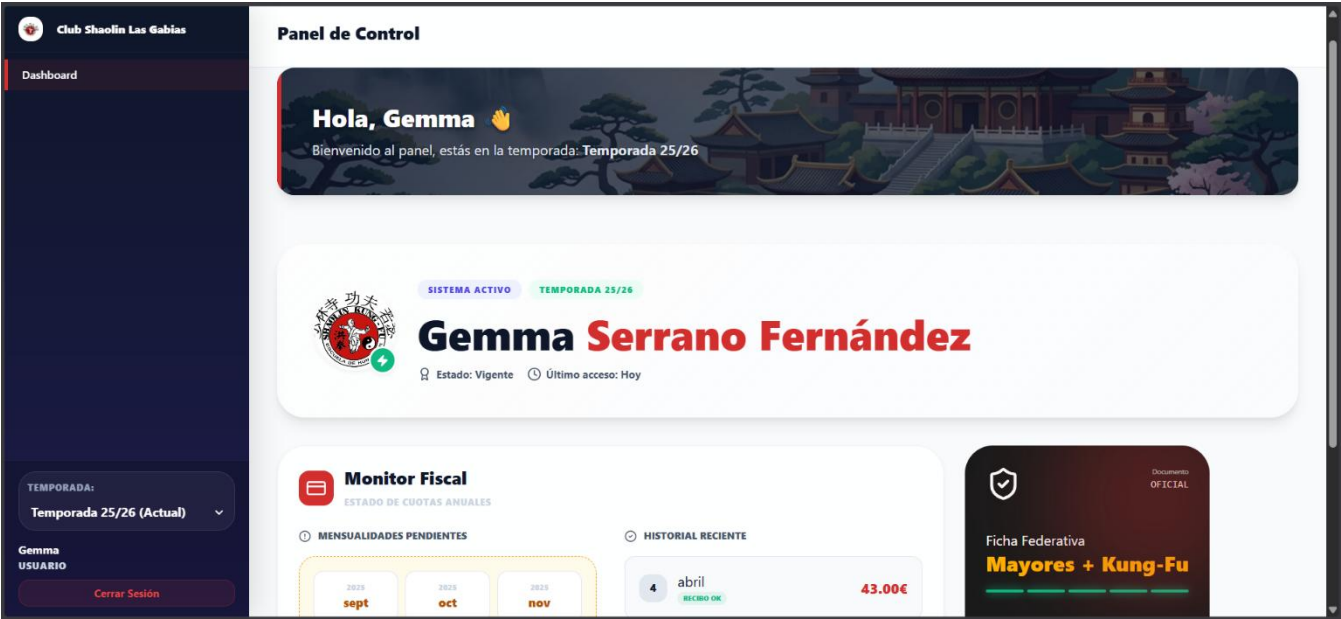
TEMPORADA: **Temporada 25/26 (Actual)**

Noelia
AYUDANTE

Cerrar Sesión

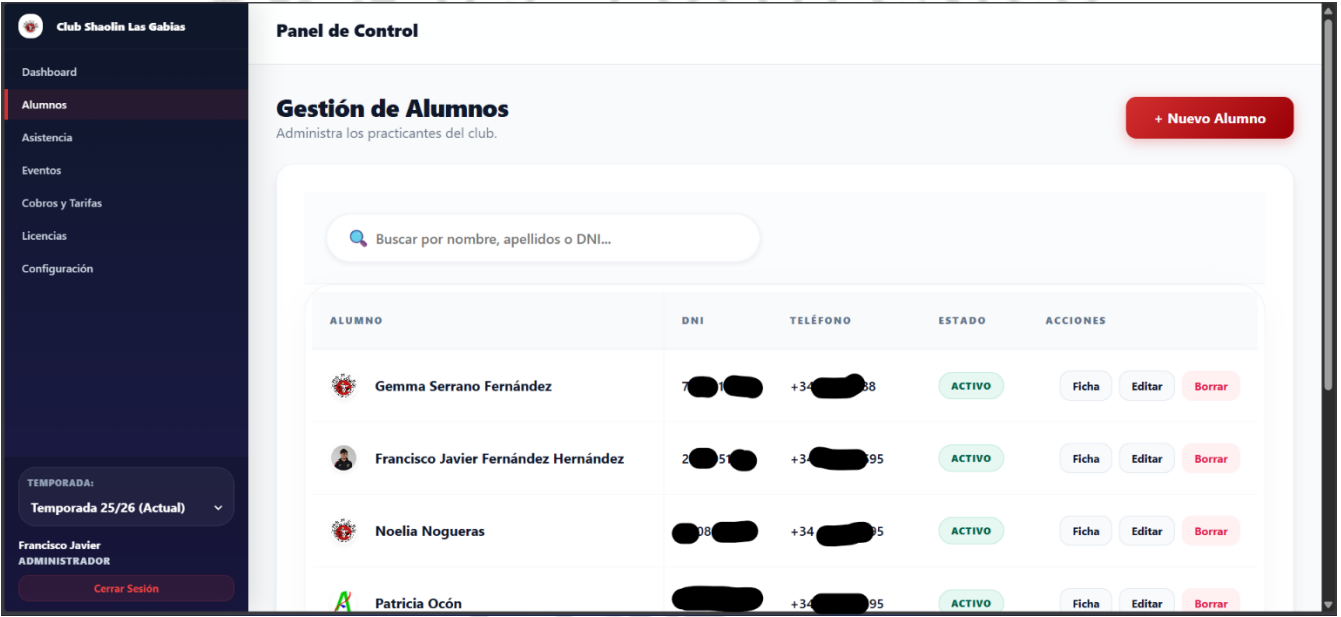
Los ayudantes son alumnos por lo que tienen información relevante suya y después la opción para Asistencias, Alumnos y Eventos

Dashboard alumno



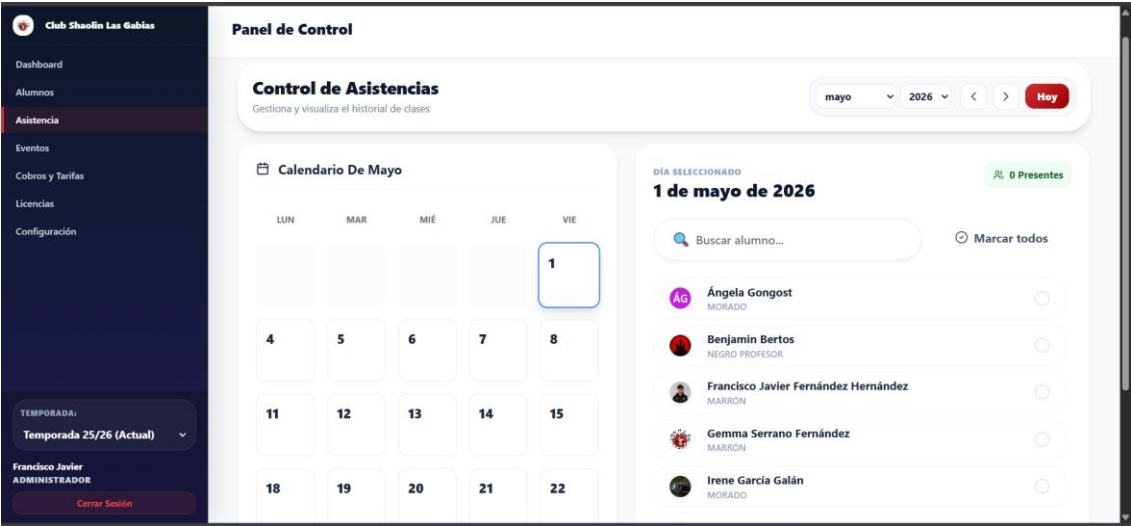
Los alumnos solo tienen su información relevante

Gestión alumnos



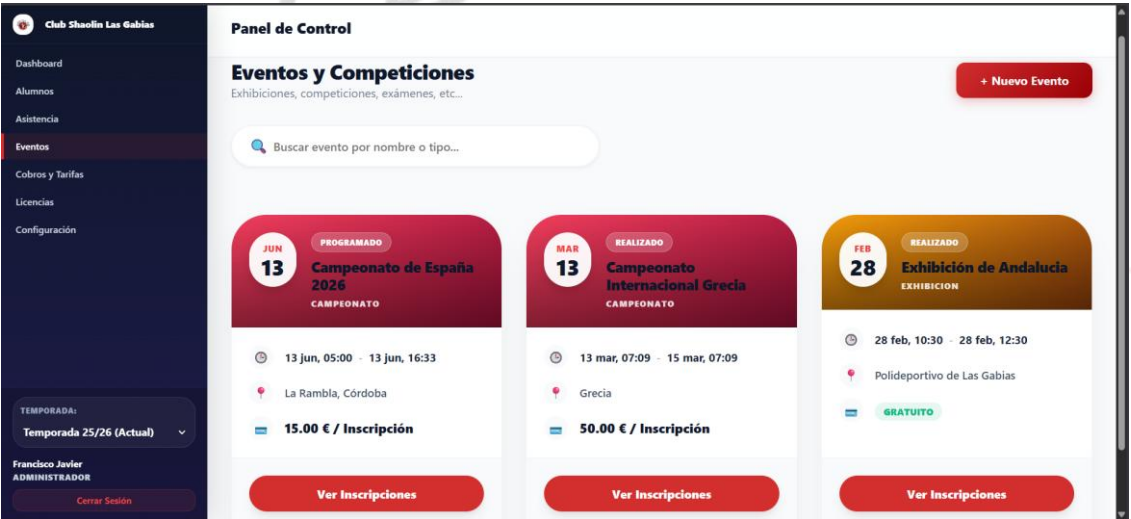
Este es el CRUD de Alumnos, además con su ficha con información importante de cada alumno

Gestión de Asistencias



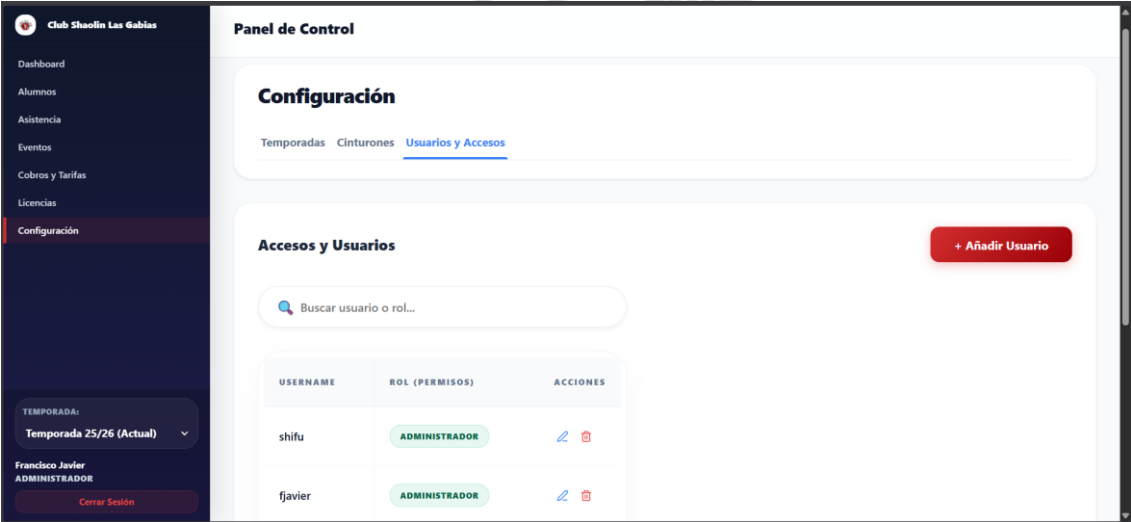
Aquí se puede ver el calendario para pasar asistencia donde solo se muestra de lunes a viernes

Gestión de Eventos



Aquí aparecen todos los eventos

Gestión de Usuarios



13. Conclusiones

El proyecto ha cumplido con los requisitos planteados inicialmente para la gestión del club, dando respuesta a las necesidades propuestas por el responsable de este.

Durante su desarrollo, se han aplicado los conocimientos y tecnologías adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web, permitiendo crear una solución funcional, eficiente y adaptada a un entorno real.

Además, la aplicación no se limita únicamente al club para el que ha sido desarrollada, sino que puede adaptarse fácilmente a otros gimnasios de artes marciales e incluso a organizaciones deportivas de distinta naturaleza.

Como posible mejora, se podría incorporar un sistema de gestión de stock para la tienda del club, permitiendo la venta de productos como ropa o material deportivo. De este modo, los alumnos podrían realizar pedidos desde la propia aplicación web, mientras que el responsable del club gestionaría dichos pedidos de forma centralizada.

Durante el desarrollo del proyecto, se han encontrado diversas dificultades. Entre ellas, destaca la complejidad del diseño de la base de datos, especialmente a la hora de asegurar que todas las entidades estuvieran correctamente relacionadas con Alumno y Temporada, al ser las tablas principales sobre las que se estructura gran parte del sistema.

Por otro lado, el diseño responsive ha requerido ajustes adicionales para garantizar un correcto funcionamiento en distintos dispositivos y tamaños de pantalla. Así mismo, la integración entre frontend y backend, junto con la gestión de autenticación mediante JWT, ha supuesto un proceso clave que ha requerido pruebas y ajustes para asegurar el correcto funcionamiento del sistema.

14. Referencias

- Aplicación web desplegada: <https://fjfh06.ddns.net/tfg>
- Repositorio en GitHub: <https://github.com/fjfh06/TFG-DAW>
- Documentación técnica adicional disponible en el archivo README.md del repositorio

La documentación adicional en el archivo README.md está en la raíz del proyecto en GitHub.